Jornada 3: Compilación, Firmware Bare-Metal y Análisis de Assembler

Daniel Rosso - Pablo Madoery

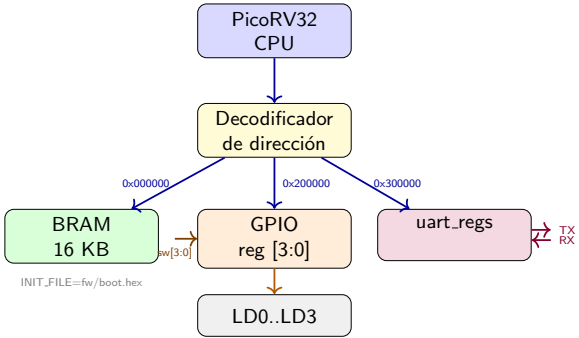
Fundación Fulgor

Miércoles 02 de setiembre de 2026

Contenidos

1. Arquitectura del SoC mínimo
2. ¿Qué es bare-metal?
3. Toolchain RISC-V
4. Secciones de memoria ELF
5. Linker script: link.ld
6. CRT0: start.S
7. Buenas prácticas: overflow de memoria
8. Flujo de compilación
9. Mapa de memoria y MMIO
10. El firmware: contador.c
11. Análisis del assembler
12. Desglose de instrucciones LUI y SW
13. Rastreo binario de una instrucción
14. Protocolo UART 8N1
15. Laboratorio

El SoC mínimo: PicoRV32 + BRAM + GPIO + UART



Mapa de memoria (lo que necesita el firmware):

- 0x000000: BRAM (16 KB) — código y datos
- 0x200000: GPIO_LEDS — LEDs [3:0] (W)
- 0x200004: GPIO_SW — switches [3:0] (R, dado)
- 0x300000: UART (dado)

RTL ya completo - el foco 100 % firmware.

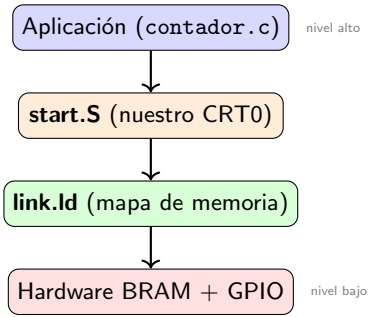
¿Qué significa "bare-metal"?

Sistema operativo convencional:

- El kernel inicializa la memoria, el stack, las interrupciones
- El loader carga el ejecutable, rellena `.data`, zeroa `.bss`
- La libc proporciona `printf`, `malloc`, `main()`
- El hardware es abstraído mediante drivers

Bare-metal en PicoRV32:

- No hay OS, no hay kernel, no hay loader
- El procesador arranca desde dirección `0x00000000`
- El stack pointer (`sp`) tiene valor **indefinido** al reset
- No hay `printf`, no hay `malloc`
- Nosotros escribimos **todo** lo que ocurre antes de `main()`



Conclusión

El primer código que ejecuta el procesador en BRAM somos **nosotros** quienes lo escribimos.

Toolchain: flags de `riscv-none-elf-gcc`

Flags esenciales de compilación:

```
riscv-none-elf-gcc          \
  -march=rv32im_zicsr \  # rv32im + CSRs
  -mabi=ilp32         \  # ABI: int/long/puntero = 32 bits
  -nostdlib           \  # sin libc ni crt0 del sistema
  -ffreestanding      \  # entorno freestanding (sin SO)
  -Os                 \  # optimizar tamaño
  -T fw/link.ld       \  # nuestro linker script
  fw/start.S fw/contador.c \ # fuentes
  -o fw/contador.elf
```

- `-mabi=ilp32`: fija el **ABI** (convención de llamada por registros, tamaño de tipos) — `int`, `long` y puntero son todos de 32 bits. Debe coincidir en **todos** los objetos que se linkean juntos
- `-nostdlib`: no linkea la libc (printf,scanf,memcpy), ni los archivos de arranque estándar (`crt0`, `crti`, `crtn`). La rutina de arranque la escribimos nosotros `start.S`
- `-ffreestanding`: le avisa al compilador que este es un entorno **freestanding** (sin sistema operativo): no puede asumir que `main` se comporta como en un programa hosted normal, ni que funciones como `memcpy`/`memset` existen con su semántica estándar

Toolchain: ABI — `-mabi=ilp32`

ABI (*Application Binary Interface*): el contrato binario entre el compilador y todo lo que se linkea con él.

- Define en qué **registros** van los argumentos y el valor de retorno de una función (en RV32: `a0`–`a7` argumentos, `a0`/`a1` retorno)
- Define el **tamaño de los tipos básicos**: `ILP32` = `Int`, `Long` y `Pointer`, los tres de 32 bits
- Define el **alineamiento** de structs y la convención de push al stack

Por qué importa

Si un objeto se compila con un ABI distinto al resto (por ejemplo `ilp32d`, que asume FPU, en vez de `ilp32`), las funciones van a interpretar mal los registros y argumentos entre sí. Esto genera un comportamiento indefinido, sin ningún error de compilación que lo avise.

¿Qué es la `libc`?

La **biblioteca estándar de C** (*C standard library*): funciones que el sistema le da al programa para no reimplementar cosas básicas.

- E/S y formato: `printf`, `scanf`, `fopen`
- Memoria dinámica: `malloc`, `free`
- Strings y memoria: `memcpy`, `memset`, `strlen`
- Control de proceso: `exit`, `abort`

Implementaciones típicas: `glibc`, `newlib`, `musl`: todas asumen que hay un **OS debajo** (heap administrado, file descriptors, etc.)

En bare-metal

No hay libc: no hay heap para `malloc`, no hay terminal/OS para que `printf` le escriba. Si querés "imprimir" algo, escribís tu propio driver de UART.

-nostdlib: crt0 / crti / crtn

CRT = *C RunTime*: objetos que la libc linkea automáticamente **antes** de correr un programa hosted normal.

- `crt0.o`: el arranque real — inicializa el stack pointer, copia `.data` a RAM, limpia `.bss`, y recién ahí llama a `main()`
- `crti.o`: prólogo de `_init`/`_fini` — corre los constructores globales de C++ (o `__attribute__((constructor))` en C) antes de `main`
- `crtn.o`: epílogo de `_init`/`_fini` — cierra lo que abrió `crti.o`

Orden normal del link (hosted): `crt0 + crti + tu_código + crtn`

Con -nostdlib

GCC no linkea ninguno de estos objetos. El rol de `crt0` lo cumple nuestro `start.S`, escrito a mano. No necesitamos `crti`/`crtn` porque no hay constructores globales de C++ que inicializar.

Toolchain: herramientas de la cadena

```
# Desensamblado completo
riscv-none-elf-objdump -d fw/contador.elf

# Extraer binario puro
riscv-none-elf-objcopy -O binary \
  fw/contador.elf fw/contador.bin

# Tamaños de secciones
riscv-none-elf-size fw/contador.elf

# Tabla de símbolos
riscv-none-elf-nm fw/contador.elf | sort
```

`nm` lista cada símbolo del ELF con su dirección, tipo (`T`=texto/código, `D`/`B`=datos inicializados/BSS) y nombre — útil para ubicar dónde quedó `main` u otra función/variable en memoria.

Toolchain: ¿por qué -march=rv32im_zicsr?

¿Por qué -march=rv32im y no rv32i?

El flag `m` activa `mul/div/rem`. Si el código usa `%` o `/` y el PicoRV32 tiene `ENABLE_DIV=0`, el procesador se cuelga silenciosamente.

El `-march` del compilador debe coincidir con los parámetros del PicoRV32.

_zicsr: GCC 14+

Las instrucciones CSR (`rdcycle`, `csrr`) requieren la extensión `Zicsr`. GCC 14 la exige explícita: `-march=rv32im_zicsr`

Secciones de memoria ELF

Sección	Contenido	En .bin	Descripción
`.text`	Código compilado	Sí	Instrucciones máquina. Solo lectura.
`.rodata`	Constantes	Sí	Strings literales, tablas de coeficientes, LUTs.
`.data`	Vars inicializadas	Sí	Globales/estáticas con valor inicial no nulo.
`.bss`	Vars sin inicializar	**No**	No ocupa espacio en el binario. `start.S` la zeroa.
Stack	Variables locales/frames	No	Crece hacia abajo. No existe en el ELF.

Ejemplo en C

```
uint32_t cnt = 42;    // .data   (init != 0)
uint32_t acc;         // .bss    (init = 0)
const uint32_t N=32; // .rodata (constante)
void main(void){...} // .text   (codigo)
```

.bss es NOBITS

El archivo `.bin`/`.elf` **no** contiene bytes para `.bss`. Solo registra su tamaño y dirección. El `start.S` escribe ceros manualmente en el rango `[_bss_start, _bss_end)`.

Linker script: `link.ld`

fw/link.ld

```
/* Mapa de memoria del soc_minimal */
MEMORY {
  BRAM (rwx) : ORIGIN = 0x000000,
               LENGTH = 16K
}

_stack_top = ORIGIN(BRAM) + LENGTH(BRAM);

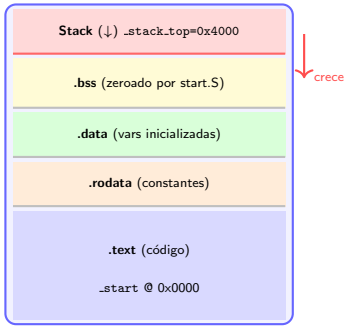
SECTIONS {
  .text : {
    *(.text.start) /* _start primero */
    *(.text*)
    *(.rodata*)
  } > BRAM

  .data : {
    _data_start = .;
    *(.data*) *(.sdata*)
    _data_end = .;
  } > BRAM

  .bss : {
    _bss_start = .;
    *(.bss*) *(.sbss*)
    *(COMMON)
    _bss_end = .;
  } > BRAM

  _firmware_end = .;
}
```

BRAM 16 KB (0x0000–0x3FFF)



Linker script: sintaxis e inicio/fin de cada sección

fw/link.ld (fragmento)

```
.data : {
  _data_start = .;        /* inicio */
  *(.data*) *(.sdata*)
  _data_end = .;          /* fin */
} > BRAM

.bss : {
  _bss_start = .;
  *(.bss*) *(.sbss*)
  *(COMMON)
  _bss_end = .;
} > BRAM

_firmware_end = .;
```

Sintaxis básica:

- `MEMORY {...}`: declara las regiones físicas disponibles (acá, una sola: `BRAM`)
- `*(.text*)`: "tomá la sección `.text` de **cualquier** archivo objeto de entrada"
- `*(.sdata*)`: variables globales pequeñas accesibles en una sola instrucción vía gp (*global pointer*), en vez de dos

¿Cómo se marca inicio y fin?

- `.` es el *contador de ubicación*: la dirección donde va lo próximo que el linker coloque
- Un símbolo como `_data_start = .;` **no reserva espacio**: solo etiqueta la dirección en ese punto
- Ponerlo **antes** de `*(.data*)` marca el inicio; ponerlo **después** marca el fin (el contador ya avanzó lo que ocupó la sección)
- `_firmware_end`, fuera de cualquier bloque de sección, marca dónde termina todo lo que el linker colocó en BRAM
- El **tamaño** de cada sección no se declara en ningún lado: sale de `_end - _start` después del link. Lo único que `MEMORY` limita es el tamaño de la región **total** (`BRAM` = 16K)

Optimización: gp y .sdata evitan el LUI

Caso general (2 instrucciones): para una dirección arbitraria de 32 bits, hace falta `lui`+`addi` (o `lui`+`lw/sw` con offset), porque el inmediato de una sola instrucción es de 12 bits.

```
lui   t0, %hi(variable)
lw    t0, %lo(variable)(t0)    # 2 instrucciones
```

Caso "chico" (1 instrucción): si la variable es global/estática y cabe en `.sdata`/`.sbss`, como acabamos de ver, GCC la ubica dentro de la ventana de ± 2 KB de gp y emite:

```
lw    t0, variable(gp)         # 1 sola instruccion
```

¿Qué es "chico"? lo define `-msmall-data-limit=N` (default `N=8` en esta toolchain):

- **Entran** ($\leq$ 8 bytes): `char`, `int`, `float`, puntero (4B en RV32), `double`/`long long` (8B, justo en el límite)
- **No entran** ($>$ 8 bytes): un `struct` de 12 bytes, `int buf[4]` (16 bytes) — quedan en `.data`/`.bss` normal, con `lui`+offset

Spoiler: `contador.c` va a seguir usando LUI para los LEDs

`GPIO_LEDS` en `0x200000` es la dirección de un **periférico MMIO**, no una variable que el **linker** coloque en `.sdata` — es una constante fija que el programador castea a puntero. El truco de gp solo aplica a datos cuyo offset respecto a gp conoce el linker, no a direcciones de hardware arbitrarias (lo vemos en detalle más adelante).

CRT0: `start.S` — el punto de entrada real

fw/start.S

```
.section .text.start
.global _start

_start:
    # 1. Inicializar stack pointer
    la   sp, _stack_top     # sp = 0x4000

    # 2. Zerear seccion .bss
    la   a0, _bss_start
    la   a1, _bss_end
_bss_loop:
    bgeu a0, a1, _bss_done
    sw   zero, 0(a0)
    addi a0, a0, 4
    j    _bss_loop
_bss_done:

    # 3. Llamar a main
    call main

    # 4. Loop infinito (main no debe retornar)
_halt:
    j _halt
```

¿Por qué necesitamos start.S?

- PicoRV32 arranca en 0x00000000: ejecuta la primera instrucción de la BRAM
- El registro sp (x2) tiene valor **indefinido** al reset
- GCC genera código que usa sp inmediatamente: el prólogo de cualquier función no-hoja hace `sw ra, -4(sp)` para guardar la dirección de retorno
- Sin `la sp, _stack_top` → escribe en dirección aleatoria → corrupción

La sección `.text.start`

El linker script coloca `*(.text.start)` como primera sección → garantiza que `_start` esté en 0x000000.

¿Entra el firmware en los 16K de BRAM?

make sections (salida real, contador.elf)

```
$ riscv-none-elf-readelf -S fw/prbs_gen.elf
 [Nr] Name     Type      Addr     Size
 [ 1] .text    PROGBITS  00000000 000232
 [ 2] .bss     NOBITS    00000234 000004

$ riscv-none-elf-nm -n fw/prbs_gen.elf
00000000 T _start
00000232 T _data_end          # .data vacía
00000234 B _bss_start
00000238 B _bss_end
00000238 B _firmware_end
00004000 B _stack_top         # ORIGIN+LENGTH(BRAM)
```

Margen disponible

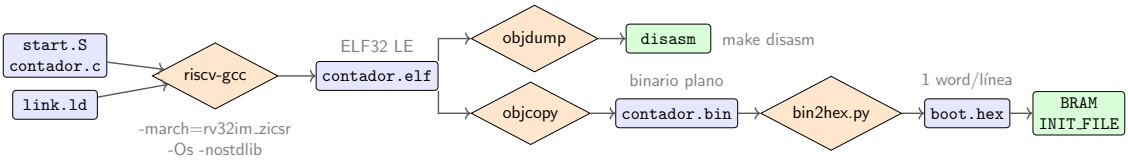
`margen = _stack_top - _firmware_end`
→ 0x4000 - 0x238 = 0x3DC8 ≈ 15.8 KB libres para pila + crecimiento de `.bss`/`.data`

`_firmware_end` no se calcula a mano: es un símbolo que `link.ld` define como `.` (contador de ubicación) justo después de `.bss` — el linker lo resuelve solo y acá simplemente lo leemos con `nm`.

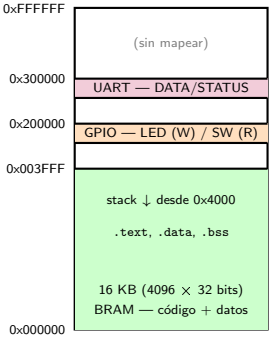
Esta BRAM no es un MCU típico:

- No hay Flash separada: `.text`, `.rodata`, `.data`, `.bss` y la **pila** conviven en la misma región de 16K
- `link.ld` **no reserva** espacio para nada: coloca las secciones una atrás de otra, cada una ocupando justo lo que necesita — sin planificación previa
- El **stack tampoco tiene tamaño fijo**: `_stack_top` es solo la dirección tope de `BRAM`; crece hacia abajo sobre lo que haya quedado libre entre `_firmware_end` y `_stack_top` — tierra de nadie compartida con el crecimiento de `.bss`/`.data`

Flujo de compilación: de C a la BRAM



Mapa de memoria y acceso a periféricos



Memory-Mapped I/O (MMIO):

- Los periféricos se acceden **como posiciones de memoria**
- El compilador no sabe esto: puede eliminar accesos "redundantes"

Sin `volatile` — peligroso:

```
uint32_t *gpio = (uint32_t*)0x200000;
*gpio = 0x5;    // el compilador puede
*gpio = 0xA;    // eliminar la 1ra linea!
```

Con `volatile` — correcto:

```
#define GPIO_LEDS \
    (*(volatile uint32_t *)0x200000)
// Cada acceso genera un lw/sw real
GPIO_LEDS = 0x5;  // no se optimiza
GPIO_LEDS = 0xA;  // ambas instrucciones
                  // quedan en el binario
```

El firmware: `contador.c` — núcleo GPIO

Ejemplo simplificado para analizar assembler paso a paso. El firmware real del lab (`fw/prbs_gen.c`) agrega la lectura de switches (y una tabla que vas a usar en la próxima jornada) sobre este mismo núcleo de GPIO + UART.

fw/contador.c (núcleo)

```c
#include <stdint.h>

#define GPIO_LEDS \
    (*(volatile uint32_t *)0x200000)

static inline uint32_t rdcycle(void) {
    uint32_t c;
    asm volatile("csrr %0, cycle":"=r"(c));
    return c;
}

#define DELAY_CYCLES  10000000UL  /* ~100ms */

void main(void) {
    uint32_t cnt = 0;
    while (1) {
        GPIO_LEDS = cnt & 0xF;
        cnt++;
        uint32_t t0 = rdcycle();
        while ((rdcycle() - t0) < DELAY_CYCLES);
    }
}
```

¿Qué genera el compilador?

- `GPIO_LEDS = cnt & 0xF`
  → cargar dirección con `lui`
  → aplicar máscara AND
  → instrucción `sw` (store word)
- `rdcycle()`
  → instrucción `csrrs rd, cycle, x0`
- Loop de delay
  → read CSR + sub + branch
- Outer loop
  → `jal zero, main+offset`

Resultado real:

- 88 bytes de código (22 instrucciones)
- Todo inlineado con `-Os`
- Tamaño: `make sections`

make disasm — salida real del compilador

riscv-none-elf-objdump -d contador.elf

```
00000000 <_start>:
  0: auipc sp,0x4     ; la sp,_stack_top
  ...                 ; (bss loop: nunca ejecuta)
 20: jal   ra,28      ; call main
 24: jal   zero,24    ; _halt: loop infinito

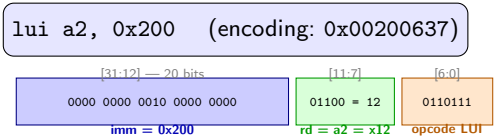
00000028 <main>:
 28: lui   a4,0x989   ; |
 34: addi  a4,a4,1663; } a4 = 9_999_999 (delay)
 2c: addi  a5,zero,0 ; cnt = 0
 30: lui   a2,0x200   ; a2 = 0x200000 (GPIO)
 38: andi  a3,a5,15   ; a3 = cnt & 0xF
 3c: sw    a3,0(a2)   ; GPIO_LEDS = a3  *** CLAVE
 40: addi  a5,a5,1    ; cnt++
 44: csrrs a1,cycle,zero ; t0 = rdcycle()
 48: csrrs a3,cycle,zero ; t  = rdcycle()
 4c: sub   a3,a3,a1   ; elapsed = t - t0
 50: bgeu  a4,a3,48   ; delay loop
 54: jal   zero,38    ; siguiente iteracion
```

Observaciones clave:

- `la sp, _stack_top` se expande a **2 instrucciones**: `auipc + addi`
- `.bss` vacío (no hay globales sin init): el loop **nunca ejecuta**
- Constante 0x200000: no cabe en 12 bits → necesita **LUI**
- El compilador **reordenó** las instrucciones (scheduling): `lui a4` antes del contador
- < 10000000 compiló a `bgeu a4, a3` con a4=9999999
- `csrr` se codifica como `csrrs rd, csr, x0`

Instrucción LUI — cargar una dirección de 32 bits

Problema: los inmediatos en instrucciones RISC-V son de 12 bits. ¿Cómo cargar 0x200000?



Lo que hace LUI:

$$\text{a2} \leftarrow \underbrace{\text{0x200}}_{\text{imm}} \times 2^{12} = \text{0x200000}$$

Los 12 bits bajos quedan en **cero**. Para otros valores: `lui + addi` forman cualquier constante de 32 bits.

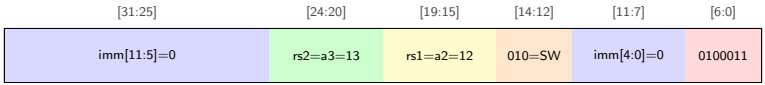
El compilador elige LUI porque:

- 0x200000 = 512 × 4096: solo tiene bits altos
- Un solo `lui` alcanza (no necesita `addi`)

Instrucción SW — encender los LEDs

El store word: `sw a3, 0(a2)` escribe a3 en `MEM[a2 + 0] = MEM[0x200000]`

`sw a3, 0(a2)` (encoding: 0x00D62023)



Formato S-type (Store):

- **rs1**: registro base de dirección (a2 = 0x200000)
- **rs2**: dato a escribir (a3 = cnt & 0xF)
- **imm**: offset dividido en 2 campos — el diseño RISC-V mantiene rs1/rs2 en posición fija para decodificación uniforme

Transacción en el bus:

- `mem_valid` = 1
- `mem_addr` = 0x200000
- `mem_wdata` = cnt & 0xF
- `mem_wstrb` = 0xF (word completa)
- → `sel_gpio` = 1
- → `gpio_reg` ← `mem_wdata[3:0]`
- → led actualizado en el **ciclo siguiente**

¿Dónde están los 32 bits de sw en el binario?

Rastreando 0x00D62023 desde objdump hasta la BRAM:

(1) `make disasm`

```
00000028 <main>:
  ...
  38: 00f7f693
      andi a3,a5,15
  3c: 00d62023
      sw a3,0(a2)
  40: ...
```

Offset 0x3C = byte 60
Word index = 60/4 = 15

(2) `xxd contador.bin`

```
0000003c: 2320 d600 ...
```

0x00D62023 en little-endian:

byte[0]	byte[1]	byte[2]	byte[3]
23	20	D6	00
LSB			MSB

(3) `boot.hex`

```
# linea 1  -> word 0 (0x00)
# linea 2  -> word 1 (0x04)
# ...
# linea 16 -> word 15 (0x3C)
00d62023
# linea 17 -> word 16 (0x40)
```

`$readmemh` carga
`BRAM[15] = 0x00D62023`

Little-endian

RISC-V es little-endian: el byte menos significativo está en la dirección más baja. 0x00D62023 → bytes en memoria: 23 20 D6 00. `boot.hex` guarda la word completa: línea 16 = 00d62023.

Herramientas de análisis: objdump -S y búsqueda de instrucciones

objdump -S — C + assembler intercalados:

```
# Compilar con -g (simbolos de debug)
$ riscv-none-elf-gcc ... -g -o contador_dbg.elf

$ riscv-none-elf-objdump -S contador_dbg.elf
...
void main(void) {
    uint32_t cnt = 0;
  28: addi  a5,zero,0     ; cnt = 0
      GPIO_LEDS = cnt & 0xF;
  38: andi  a3,a5,15      ; a3 = cnt & 0xF
  3c: sw    a3,0(a2)      ; MEM[a2] = a3
      cnt++;
  40: addi  a5,a5,1       ; cnt++
```

Buscar instrucciones problemáticas:

```
# Si el firmware usa division/resto:
$ riscv-none-elf-objdump -d fw.elf \
    | grep -E "rem|div|divu|remu"
# Si aparece: necesitas ENABLE_DIV=1
# Si no aparece: ENABLE_DIV=0 es OK

# Si hay punto flotante (NO soportado):
$ riscv-none-elf-objdump -d fw.elf \
    | grep -E "fadd|fmul|fdiv|fsqrt"
# Usar -march=rv32im (sin 'f') para evitar
```

Generar listing completo:

Un listing es un archivo de texto con el código fuente y su assembler intercalados línea por línea (como la salida de `objdump -S` de arriba)

```
$ riscv-none-elf-objdump -lSd contador.elf \
    > firmware.lst
# firmware.lst: referencia para depuración
# -l: agrega path y numero de linea C
```

Target en el Makefile

```
disasm-src: CFLAGS += -g
disasm-src: fw/contador.elf
    $(OBJDUMP) -S -d $<
```

Protocolo UART 8N1

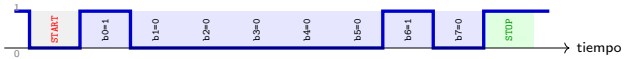
Características

- **Asíncrono**: sin señal de reloj compartida
- **8N1**: 8 bits de datos, sin paridad, 1 stop bit
- **Idle**: línea en alto (1) en reposo
- LSB se transmite primero

Uso	Baud	Tiempo/bit
Arty A7 (real)	115 200	8.68 µs
Simulación	1 000 000	1 µs

START: baja la línea para sincronizar. **STOP**: restaura a reposo. El receptor (`uart_rx.v`, ya dado) muestrea al centro de cada bit.

Trama serial — `'A' = 0x41` (LSB primero)



El módulo `uart_tx.v`/`uart_rx.v` que serializa/deserializa esta trama ya está resuelto — es la "caja negra" de hardware sobre la que escribimos el driver.

Mapa de registros UART

Dos registros de 32 bits (base `0x00300000`)

Offset	Nombre	Función
+0x00	`DATA`	W: byte a TX / R: byte RX
+0x04	`STATUS`	bits de estado

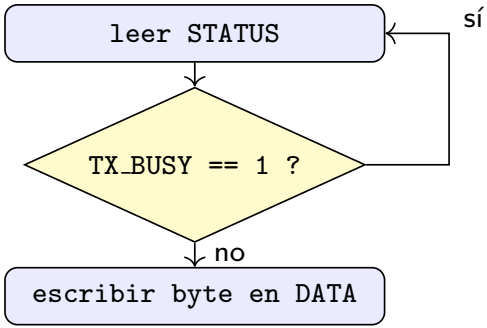
Bits del registro STATUS

Bit	Nombre	Significado
0	`TX_BUSY`	TX en curso (esperar antes de escribir)
1	`RX_VALID`	byte recibido disponible en DATA
2	`RX_OVERFLOW`	se perdió un byte RX

Sin FIFO

TX/RX tienen **un solo byte** de buffer (`tx_data_r`/`rx_latch`), no una cola — de ahí `RX_OVERFLOW`. El ACK por chunk de la Jornada 5 responde en parte a esta limitación.

Protocolo de transmisión (TX)



Este diagrama **es** `uart_putc`: el driver que vamos a completar en el lab.

El driver: `uart.c` — lo que falta escribir

fw/uart.c

```c
#define UART_BASE    0x00300000UL
#define UART_DATA    (*(volatile uint32_t*)UART_BASE)
#define UART_STATUS  (*(volatile uint32_t*)(UART_BASE+4))
#define UART_TX_BUSY (1u << 0)

uint32_t uart_tx_count = 0;

static void uart_wait_tx(void)
{
    // TODO 1: esperar mientras
    //   UART_STATUS & UART_TX_BUSY
}

void uart_putc(char c)
{
    uart_wait_tx();
    UART_DATA = (uint32_t)(uint8_t)c;
    uart_tx_count++;
}

void uart_print_uint(uint32_t v)
{
    char tmp[11]; int i = 0;
    if (v == 0) { uart_putc('0'); return; }
    // TODO 2: extraer digitos con v%10,
    //   dividir v/=10, guardar en tmp[]
    while (i--) uart_putc(tmp[i]);
}
```

Por qué `volatile`

- Sin él, el compilador puede leer `STATUS` una sola vez y "optimizar" el resto del loop
- El hardware cambia `TX_BUSY` de forma asíncrona: C necesita releer *cada vez*

TODO 1 — busy-wait

Mismo patrón de handshake que `mem_ready` en el bus, pero visto desde C: "no avances hasta que el hardware esté listo".

TODO 2 — decimal desde binario

No existe una instrucción que convierta un entero a texto. Se arma dígito a dígito con `% 10` (van quedando en orden invertido) y se imprime al revés.

`static` y `extern`: ahora `uart` es `.c`+`.h`

`uart_wait_tx` es `static`: enlace interno, invisible fuera de `uart.c` — detalle de implementación, no del contrato. `uart_tx_count` se define sin `static` en `uart.c` y se declara `extern` en `uart.h`: así `prbs_gen.c` la lee sin volver a definirla.

Jornada 3 — Actividades del estudiante

Estructura del lab:

- `fw/prbs_gen.c` — 2 TODOs:
  - TODO 1: Definir `GPIO_LEDS` con `volatile`
  - TODO 2: Escribir `block_num & 0xF` en los LEDs
- `fw/uart.h` — 2 TODOs (driver UART):
  - TODO 1: busy-wait en `uart_putc` (`TX_BUSY`)
  - TODO 2: extracción de dígitos en `uart_print_uint`
- `fw/sw.h` — 3 TODOs (driver de switches, mismo patrón MMIO que GPIO_LEDS pero de lectura):
  - TODO 1: puntero volátil `GPIO_SW` en `0x200004`
  - TODO 2: `sw_read()` — enmascarar con `0xF`
  - TODO 3: completar 3 valores de `ERR_PERIOD_TABLE`
- Dado (sin editar): `prbs.h` (generador LFSR), `start.S`, `link.ld`, `rtl/soc_minimal.v` completo (BRAM + GPIO + switches + UART ya conectados, igual que en la Jornada 2), testbench, TCLs

Flujo de trabajo:

```
source ../../env.sh      # toolchain en PATH
make check               # verificar herramientas
# editar fw/prbs_gen.c y fw/uart.h
make hex-sim             # compilar con -DSIM
make sim                 # simular con iverilog
make disasm              # analizar assembler
make sections            # tamaños de secciones
make hex synth           # para la placa real
make program             # programar Arty A7
```

Salida esperada de `make sim`:

```
=== SoC arrancado ===
=== Jornada 3: GPIO + switches + UART ===
Move SW[3:0] y mira como cambia
    el valor decodificado

[bloque 0] SW=8 decod=1021
  [GPIO wr] ciclo=133842 wdata=0x1 led=0000
[bloque 1] SW=8 decod=1021
  [GPIO wr] ciclo=162744 wdata=0x2 led=0001
...

[OK] 4 escritura(s) al GPIO
[OK] 212 caracter(es) por UART -- SoC funcional
```

El banner y cada línea `[bloque N] SW=X decod=Y` llegan por UART; es el mismo firmware (y el mismo `SW=8` fijo en el testbench) que ya viste correr en la Jornada 2 — ahora lo escribiste vos. Para qué sirve `decod` lo vas a ver en la Jornada 4.

Recapitulación

Lo que aprendimos:

- **Bare-metal**: sin OS, `start.S` inicializa todo antes de `main()`
- **start.S**: inicializa sp, zeroa `.bss`, llama a `main()`
- **link.ld**: ubica secciones en memoria, define `_stack_top`
- **Secciones ELF**: `.text` / `.data` / `.bss` / `.rodata`
- **LUI**: carga los 20 bits altos de una dirección de 32 bits
- **SW (S-type)**: el inmediato está dividido para posición fija de rs1/rs2
- **volatile**: obliga al compilador a generar el acceso a memoria
- **static / extern**: `static` da enlace interno (invisible fuera del `.c`); `extern` declara en el `.h` lo que se comparte entre archivos
- **make disasm**: herramienta de análisis esencial en embedded
- **UART 8N1**: protocolo asíncrono, registros `DATA`/`STATUS` mapeados en memoria
- **Driver UART**: busy-wait sobre `TX_BUSY` + conversión de enteros a texto dígito a dígito
- **Driver de switches**: mismo patrón MMIO que los LEDs, pero de lectura
- **Overflow de memoria**: `make sections` para chequear margen entre `_firmware_end` y `_stack_top`

Checklist final: TODOs del laboratorio

`fw/prbs_gen.c`

- TODO 1: `GPIO_LEDS` con `volatile`
- TODO 2: escribir `block_num & 0xF` en los LEDs

`fw/uart.h`

- TODO 1: busy-wait en `uart_putc` (`TX_BUSY`)
- TODO 2: extracción de dígitos en `uart_print_uint`

`fw/sw.h`

- TODO 1: puntero volátil `GPIO_SW` en `0x200004`
- TODO 2: `sw_read()` enmascarando con `0xF`
- TODO 3: completar 3 valores de `ERR_PERIOD_TABLE`

7 TODOs en total — el resto del proyecto (`start.S`, `link.ld`, RTL, testbench) ya está dado.